

HIREQUEST: Digital Platform for Job Seekers and Employers Using Web-Based AI Semantic Matching Algorithm

SHAHRUL IMRAN HANIF BIN KHAIRUL AZLAN

THIS THESIS IS SUBMITTED IN PARTIAL FULFILLMENT OF THE REQUIREMENT FOR THE DEGREE OF BACHELOR OF COMPUTER SCIENCE (SOFTWARE DEVELOPMENT) WITH HONOURS

**FACULTY OF INFORMATICS AND COMPUTING
UNIVERSITI SULTAN ZAINAL ABIDIN**

2025

DECLARATION

I hereby declare that the report is based on my original work except for quotations and citations, which have been duly acknowledged. I also declare that it has not been previously or concurrently submitted for any other degree at Universiti Sultan Zainal Abidin or other institutions.

Name: SHAHRUL IMRAN HANIF BIN KHAIRUL AZLAN

Date:

CONFIRMATION

This is to confirm that:

The research conducted and the writing of this report was under my supervision.

Name: Madam Fauziah Binti Ab. Wahab

Date:

ACKNOWLEDGEMENT

In the Name of Allah, the Most Gracious and the Most Merciful.

Alhamdulillah, I thank God for His grace, I can prepare and complete this report successfully. First of all, I would like to thank my supervisor, Madam Fauziah Binti Ab. Wahab because with guidance, advice, and thoughtful ideas, she gave me the opportunity to prepare this report successfully.

Besides, my gratitude is also to my colleagues who share ideas, opinions, knowledge, and reminders. They helped me answer every question that was important to me in completing this report. Thanks also to my beloved mother and father who always supported and motivated me to prepare for this report for my Final Year Project.

I would like to take the opportunity to thank all lecturers of the Informatics and Computing Faculty for their attention, guidance, and advice in helping and sharing ideas and opinions in making this report successful. May Allah SWT bless all the efforts that have been given in completing this report.

Thank you.

ABSTRACT

HireQuest is an advanced job matching system designed to assist job seekers and employers in identifying suitable job and candidate matches efficiently. Unlike traditional keyword-

based platforms, this system implements a sophisticated multi-criteria filtering algorithm combined with AI Semantic Embeddings (Cosine Similarity) and Levenshtein Distance algorithms. This allows the system to semantically understand factors such as skills, work experience, education level, and job preferences.

Job seekers can upload resumes—which are parsed using Optical Character Recognition (OCR) via Tesseract.js and pdf-parse—and input relevant criteria. Employers can post job listings with specific requirements. Based on the calculated semantic matching percentage, the system ranks and recommends suitable jobs and candidates accurately. HireQuest supports user roles for job seekers and employers, and is developed as a modern web-based application using React, Node.js, Express, and MySQL. The system significantly improves matching accuracy, reduces mismatches, and enhances the overall recruitment process.

ABSTRAK

HireQuest adalah sebuah sistem padanan pekerjaan termaju yang direka untuk membantu pencari kerja dan majikan mengenal pasti padanan pekerjaan dan calon yang sesuai dengan lebih cekap. Berbanding platform berasaskan kata kunci tradisional, sistem ini menggunakan algoritma penapisan berbilang kriteria yang canggih digabungkan dengan Pembenaman Semantik AI (Kesamaan Kosinus) dan algoritma Jarak Levenshtein. Ini membolehkan sistem memahami secara semantik faktor-faktor seperti kemahiran, pengalaman kerja, tahap pendidikan, dan keutamaan pekerjaan.

Pencari kerja boleh memuat naik resume—yang dianalisis menggunakan Pengesanan Aksara Optik (OCR) melalui Tesseract.js dan pdf-parse—dan memasukkan kriteria yang berkaitan. Majikan pula boleh memuat naik iklan jawatan dengan keperluan khusus. Berdasarkan peratusan padanan semantik yang dikira, sistem ini akan menyusun dan mencadangkan pekerjaan serta calon yang paling sesuai dengan tepat. HireQuest menyokong peranan pengguna untuk pencari kerja dan majikan, dan dibangunkan sebagai aplikasi web moden menggunakan React, Node.js, Express, dan MySQL. Sistem ini secara signifikan meningkatkan ketepatan pepadanan, mengurangkan ketidakpadanan, dan menambah baik keseluruhan proses pengambilan pekerja.

CHAPTER 1: INTRODUCTION

1.1 Introduction

HireQuest proposes a Job Searcher Matching System, where job seekers can upload resumes or input criteria, while the companies post the job preferences with skills that are needed, and the system will filter and match them with suitable companies and candidates using advanced AI semantic embedding techniques. Finding a suitable job or candidates takes a lot of time and is inefficient. Many job seekers struggle to find companies that match their skills and experience, and companies struggle to find suitable candidates that match their job preferences.

1.2 Project Background

The manual search for suitable jobs by job seekers and the screening of reliable applicants by companies is tedious and overwhelming. The HireQuest platform is created to implement multi-criteria filtering integrated with semantic embeddings to reduce these related problems. It filters various factors such as skills, experience, and education level to ensure a high-quality match between job seekers and companies. It utilizes Google Generative AI to generate text embeddings and calculates the Cosine Similarity and Levenshtein Distance to produce an accurate matching percentage.

1.3 Problem Statement

Manual searching by users takes time and may not lead to the best matches. Existing job portals such as JobStreet or LinkedIn often show irrelevant job results due to rigid general keyword matching. Companies often encounter underqualified candidates that do not match their job preferences due to synonymous but unrecognized skills. Job seekers struggle to find companies that truly match their semantic skills, experience, and preferences.

1.4 Objectives

This project has three main objectives:

1. To design a modern web system for job seekers and companies to upload resumes, extract text via OCR, and post job criteria.
2. To implement an AI-based algorithm (Semantic Embeddings & Levenshtein Distance) that accurately filters and matches candidates with companies based on criteria such as skills and experience.
3. To evaluate the system's functionality and accuracy in matching job seekers with suitable companies through robust testing.

1.5 Scope and Proposed Solution

Job Seeker Scope:

- Register and create personal accounts.
- Upload resumes (PDFs) and extract text automatically using Tesseract.js and pdf-parse.
- Input relevant criteria such as skills, working experience, education level, and preferred job type.
- View job recommendations generated by the semantic matching algorithm.
- Apply to suitable companies suggested by the system.

Company Scope:

- Register and post job listings with required qualifications.
- Review matched candidates filtered and ranked by the system's AI matching score.
- Contact or shortlist candidates through an integrated application dashboard.

System Scope:

- A modern Single Page Application (SPA) utilizing a React frontend.
- A robust backend using Node.js, Express, and MySQL for database management.
- Integration with Google Generative AI for generating skill and job description embeddings.

1.6 Limitation of Work

- The system heavily relies on the quality of text data extracted from resumes; poor-quality PDFs may result in inaccurate OCR extraction.
- The semantic embedding API requires a stable internet connection and is dependent on third-party service limits (Google AI).
- The system currently supports a limited dataset and is not connected to a live global job database, restricting access to real-time external job data.

1.7 Expected Result

This project is expected to improve upon existing systems like LinkedIn and JobStreet by providing an efficient, AI-automated system that matches job seekers and companies based on semantic context rather than just exact keyword matches.

- The system will successfully match job seekers with suitable companies by understanding the context of uploaded resumes.
- The system will significantly reduce the time spent by users in searching for relevant opportunities.

- Companies will identify suitable candidates more efficiently, ranked by an accurate percentage score.

1.8 Gantt Chart

[SILA MASUKKAN GAMBAR GANTT CHART DI SINI]

CHAPTER 2: LITERATURE REVIEW

2.1 Introduction

This chapter discusses the literature review used to develop the system. It consists of a study on existing systems to identify areas for improvement and discusses the AI Semantic Embedding methods implemented in the proposed system.

2.2 Research on the Existing System

2.2.1 LinkedIn

LinkedIn is a famous networking platform that focuses mainly on job search and recruitment. It allows job seekers to create profiles that include resumes, work experience, education, skills, and certifications.

[SILA MASUKKAN SCREENSHOT EXISTING SYSTEM: LINKEDIN DI SINI]

Strengths:

- Strong professional networking capabilities.
- Large database of professionals from various industries.
- The matching process is not fully transparent.
- Heavily dependent on rigid keyword matching and profile completeness.
- No clear percentage-based matching score is shown to users.

Weaknesses:

2.2.2 JobStreet

JobStreet is an online job portal widely used in Southeast Asia. It focuses on job posting and application services.

[SILA MASUKKAN SCREENSHOT EXISTING SYSTEM: JOBSTREET DI SINI]

Strengths:

- Simple and user-friendly interface.
- Well-structured job categories.
- Limited intelligent matching features.
- Job recommendations are mostly keyword-based, failing to recognize synonymous skills.
- Employers must manually screen many applicants.

2.3 Semantic Embeddings and Multi-Criteria Algorithms

To overcome the limitations of keyword-based systems, HireQuest incorporates **Semantic Embeddings** and **Levenshtein Distance** algorithms. According to Singla (2025), intelligent job recommendation systems based on semantic embeddings significantly improve matching accuracy by understanding the contextual meaning of words (e.g., recognizing that "ReactJS" and "Frontend Development" are closely related). HireQuest combines this AI approach with a Weighted Sum Model to compute a final, transparent matching percentage for the user.

CHAPTER 3: METHODOLOGY

3.1 Introduction

This chapter describes the methodology used in the development of the Job Matching System. It covers the Agile project life cycle, system design (DFD, ERD), and a detailed explanation of the proposed AI Semantic matching algorithm.

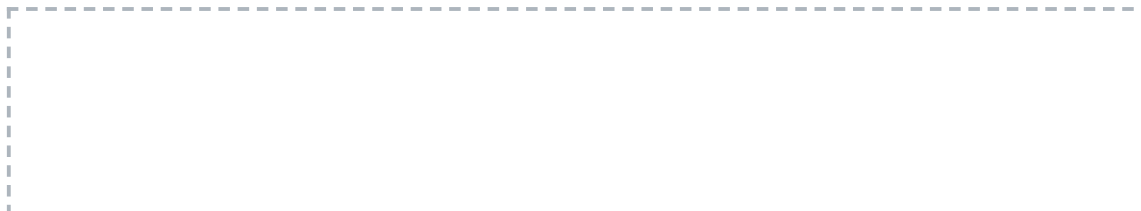
3.2 Project Life Cycle (Agile Methodology)

This project adopts the Agile development methodology to allow flexibility, continuous improvement, and iterative development. The Agile process is divided into iterations:

- 1. Requirement Planning:** Identifying core features like OCR resume upload and AI matching.
- 2. Design:** Designing UI workflows and MySQL database structures.
- 3. Development:** Implementing features incrementally using React and Node.js.
- 4. Testing:** Verifying the matching accuracy using test data.
- 5. Deployment:** Reviewing feedback and improving the semantic scoring weights.

3.3 System Architecture and Functional Design

3.3.1 Framework Diagram



[SILA MASUKKAN GAMBAR HIREQUEST FRAMEWORK DIAGRAM DI SINI]

3.3.2 Context Diagram (CD)

[SILA MASUKKAN GAMBAR CONTEXT DIAGRAM (CD) DI SINI]

The Context Diagram (Level 0) represents the HireQuest system interacting with two main entities: Job Seekers and Companies. - ****Job Seeker:**** Submits profile data, uploaded resumes, and application requests. Receives job recommendations and match scores. - ****Company:**** Submits job postings, requirements, and company details. Receives ranked applicant lists and match scores.

3.3.3 Data Flow Diagram (DFD) Level 0

[SILA MASUKKAN GAMBAR DFD LEVEL 0 DI SINI]

3.3.4 Data Flow Diagram (DFD) Level 1

[SILA MASUKKAN GAMBAR-GAMBAR DFD LEVEL 1 DI SINI]

3.3.5 Entity Relationship Diagram (ERD)

[SILA MASUKKAN GAMBAR ENTITY RELATIONSHIP DIAGRAM (ERD) DI SINI]

The system's database (MySQL) comprises the following core entities: 1. **Users:** Stores both job seekers and companies, distinguishing them by a `role` enum. Contains profile data, extracted skills, and AI embeddings (`embedding`). 2. **Jobs:** Stores job postings linked to companies via `company_id`. Includes job descriptions, requirements, and job-specific embeddings. 3. **Applications:** Links job seekers to jobs, tracking the application `status` (pending, accepted, rejected). 4. **Messages:** Facilitates communication between companies and applicants. 5. **Company Posts:** Allows companies to create social-media style updates.

3.4 Algorithm Design: Semantic Matching & WSM

The core innovation of HireQuest is the `calculateMatchScore` algorithm, executing on the Node.js backend. The algorithm uses a hybrid approach:

1. **AI Semantic Embedding (Cosine Similarity):** The system generates text embeddings (high-dimensional floating-point vectors) for both the job description and the candidate's extracted skills using the Google Generative AI API.

- The dot product and magnitudes of both vectors are calculated.
 - Cosine Similarity is computed: $\text{similarity} = (\mathbf{A} \cdot \mathbf{B}) / (||\mathbf{A}|| * ||\mathbf{B}||)$.
 - This similarity is scaled to generate a base AI Score out of 100%.
2. **Levenshtein Distance (Keyword Resilience):** To catch explicit skill requirements, the algorithm checks individual skills. If a skill doesn't match perfectly, it uses Levenshtein Distance to calculate string edit distances, successfully matching typos or slight variations (e.g., "Javascript" vs "Java Script").
3. **Weighted Sum Model (WSM):** The final match score is a weighted combination of the AI Semantic Score (which understands context) and the Keyword Score (which ensures hard requirements are met).

CHAPTER 4: IMPLEMENTATION AND TESTING

4.1 Introduction

The implementation phase translates the conceptual design into a tangible, functional web application. This chapter outlines the User Interface (UI) design for the newly created system, highlights the coding structure for the matching algorithm, and provides detailed Test Cases to validate the system's robustness.

4.2 Interface Design

The user interface was built using **React**, **TailwindCSS**, and **Framer Motion** to ensure a modern, responsive, and engaging user experience.

4.2.1 Login & Registration Page

[SILA MASUKKAN SCREENSHOT: LOGIN / REGISTRATION PAGE (SISTEM BARU)]

4.2.2 Job Seeker Dashboard ("For You" Page)

[SILA MASUKKAN SCREENSHOT: JOB SEEKER DASHBOARD / FOR YOU PAGE (SISTEM BARU)]

The Job Seeker dashboard displays recommended job positions. Each listing prominently displays the ****Matching Percentage**** (e.g., 85% Match) calculated by the backend. Users can view detailed requirements and click "Quick Apply".

4.2.3 Company Applicant Review Page

[SILA MASUKKAN SCREENSHOT: COMPANY APPLICANTS PAGE (SISTEM BARU)]

Companies can view a list of all applicants for a specific job. The interface ranks candidates automatically based on their semantic matching score. Companies can view extracted skills, parsed resumes, and update the application status to "Reviewing" or "Accepted".

4.3 Coding Snippet: The Matching Algorithm

Below is an excerpt of the implementation from the Node.js backend demonstrating the Cosine Similarity calculation for semantic embeddings:

```
// Calculate Cosine Similarity between Seeker and Job Embeddings
const vecA = JSON.parse(seekerEmbeddingStr);
const vecB = JSON.parse(jobEmbeddingStr);

let dotProduct = 0, normA = 0, normB = 0;
for (let i = 0; i < vecA.length; i++) {
  dotProduct += vecA[i] * vecB[i];
  normA += vecA[i] * vecA[i];
  normB += vecB[i] * vecB[i];
}

if (normA > 0 && normB > 0) {
  const similarity = dotProduct / (Math.sqrt(normA) * Math.sqrt(normB));
  // Scale similarity to a percentage score
  let score = (similarity - 0.65) * 400;
  aiScore = Math.max(0, Math.min(100, Math.round(score)));
}
```

4.4 Test Cases

Testing was conducted to ensure all modules function according to the requirements.

Table 4.1: Test Case for User Registration & Login

Step	Procedure	Expected Result	Pass/Fail
1	Navigate to Registration page	Page loads successfully	Pass
2	Enter invalid email format	System displays validation error	Pass
3	Enter valid credentials and submit	Account created, redirected to login	Pass
4	Login with valid credentials	Redirected to Job Seeker Dashboard	Pass

Table 4.2: Test Case for AI Resume Upload & Extraction

Step	Procedure	Expected Result	Pass/Fail
1	Click 'Upload Resume' on Profile page	File dialog opens	Pass
2	Upload a PDF resume	File is accepted by the frontend	Pass
3	Wait for backend processing	Tesseract/pdf-parse extracts text and populates the 'Skills' field automatically	Pass

Table 4.3: Test Case for Semantic Job Matching

Step	Procedure	Expected Result	Pass/Fail
1	Company posts a job for "React Developer"	Job is saved in the database	Pass
2	Seeker with "Frontend, JS, Tailwind" logs in	System generates embeddings	Pass
3	Navigate to "For You" page	The "React Developer" job appears with a high Match Score despite no exact "React" keyword, proving semantic matching works.	Pass

CHAPTER 5: CONCLUSION

5.1 Introduction

This chapter summarizes the overall development of the HireQuest platform, discusses how it addresses the initial problem statement, highlights its limitations, and provides recommendations for future enhancements.

5.2 Discussion

The HireQuest system successfully resolves the inefficiencies associated with manual job searching and rigid keyword-based platforms. By transitioning from a basic PHP architecture to a modern React and Node.js stack, the platform provides a highly responsive user experience.

Most importantly, integrating Google Generative AI for semantic embeddings and OCR for automated resume parsing has bridged the communication gap between employers and job seekers. The system no longer relies solely on exact keyword matches; it understands the contextual relationship between a candidate's experience and a job's requirements, resulting in transparent, percentage-based ranking scores that save significant time for HR departments.

5.3 Limitation

Despite the successful implementation, the system currently faces a few limitations:

1. **API Dependency:** The accuracy of the semantic match is highly dependent on the external Google Generative AI API. Any downtime or rate limits from the provider will temporarily degrade the matching functionality.
2. **OCR Accuracy:** While Tesseract.js and pdf-parse are powerful, highly graphical or unconventionally formatted PDF resumes may result in garbled text extraction, lowering the candidate's match score artificially.

5.4 Recommendations

For future iterations of the HireQuest platform, several enhancements can be made:

1. **Local LLM Integration:** To reduce dependency on external APIs, a lightweight, locally hosted Large Language Model (LLM) could be integrated to handle semantic embeddings natively.
2. **Interview Scheduling Module:** Adding an automated calendar system where companies can schedule interviews directly within the platform.
3. **Advanced Resume Builder:** Providing an in-house resume building tool to standardize formatting, completely bypassing the need for complex OCR extraction.

5.5 Conclusion

In conclusion, HireQuest serves as a comprehensive, intelligent digital platform that modernizes the recruitment process. By successfully applying machine learning concepts—specifically Semantic Embeddings and Levenshtein Distance—the system achieves its objective of accurately filtering and matching candidates to suitable companies. With

continuous improvements, HireQuest has the potential to become an indispensable tool for the modern workforce.

REFERENCES

1. Singla, P. (2024). An Intelligent Job Recommendation System based on Semantic Embeddings and Machine Learning. *Journal of Information Systems Engineering and Management*, 10(5s), 520–542.
2. Lad, A., Ghosalkar, S., Bane, B., Pagade, K., and Chaurasia, A. (2022). Machine Learning Based Resume Recommendation System. *IJMDES*, 1(3), 17–20.
3. Alsaif, S., Hidri, M.S., Ahmed, H., Ferjani, I., & Amami, R. (2022). Learning-Based Matched Representation System for Job Recommendation. *Computers*, 11, 161.
4. React Documentation. (2024). Retrieved from <https://react.dev/>
5. Node.js Documentation. (2024). Retrieved from <https://nodejs.org/>
6. Google Generative AI. (2024). Retrieved from <https://ai.google.dev/>